

《函数式编程与编译原理》大作业说明

一、作业目标说明：

- **目标：**在函数式思维训练与编译底层理解两个维度上，建立“优雅抽象”与“机械本质”的对照认知。禁止提交“黑箱代码”，强调可解释的独立实现与显式的大模型使用声明。
- **能力锚点：**
 - 任务一：递归模式识别、不可变性代价感知、尾递归优化验证
 - 任务二：指令级语义映射、控制流图还原、抽象语法树与字节码对应关系
- **学术诚信红线：**代码相似度检测+大模型使用日志双重校验。检出未声明 AI 协助或提示词造假的，作业计零分。

二、任务细则（二选一，完成其中一个任务即认定为完成课程大作业）

- **任务一：函数式算法实现与运行时解构**
 - **核心要求：**
 - ◆ **语言：**Haskell/OCaml/Scala/F#（纯函数式子集）或 Python（强制只使用 `map/filter/reduce/recursion`，禁止赋值语句）
 - ◆ **算法复杂度：**必须包含显式的 $O(n^2)$ 嵌套结构（递归深度或高阶函数链）
 - ◆ **注释规范：**每 5 行代码至少 1 条注释，需说明“此处若用命令式语言会如何实现”
 - **交付三件套：**源代码（`.hs/.ml/.py`），包含手工编写的测试驱动程序（禁止仅用 QuickCheck 等生成式测试）；运行时对比报告（尽可能简明），至少

指出 1 处函数式程序特有的运行时行为（如惰性求值延迟计算、不可变数据复制开销、尾递归优化的汇编级证据）；大模型使用声明表（模板见附录）

➤ 任务二：Python 字节码反编译与语义等价性论证

■ 核心要求：

- ◆ 源代码：手工编写 $O(n^2)$ 复杂度的 Python 程序（禁止直接反编译他人代码）
- ◆ 工具链：dis 模块（或其他反汇编工具）+ uncompyle6/decompyle3/pycdc（或其他等效工具）
- ◆ 反编译深度：必须逐函数标注，至少完成 1 个核心函数的完整字节码注释（逐条 OPCODE）

- 交付三件套：源代码（.py）与字节码截图（python -m dis your_script.py 输出）；反编译复原代码（工具生成）与手工人机混合优化版本（需说明为何比工具版本更易读，为何手写源代码与反编译版本等价）；大模型使用声明表（模板见附录）

附录：大模型使用声明表模板

AI 辅助声明表

1. 使用场景（可多选，在选项开头打勾并附于所提交报告末尾）

- ☐ 初始算法选型咨询

- ☐ 语法错误调试（附：错误截图）

-[] 运行时概念解释（如“什么是 Haskell 的 WHNF”）

-[] 字节码指令功能确认（附：具体 OPCODE）

-[] 报告结构润色

-[] 其他：_____

2. 提示词原文（精确复制所使用的所有提示词，并按照使用顺序排列）